

0.25.3. La fonction `deplace()` — avancer sur les deux axes

`#include <deplace>` : `deplace(x, y, tuile, pasX, pasY)` fait avancer la tuile en diagonale, puis tout droit.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `deplace(x, y, tuile, pasX, pasY)` fait avancer la tuile sur **les deux axes** en même temps : `pasX` vers la droite (+) ou la gauche (-), `pasY` vers le bas (+) ou le haut (-). 5 et 5 : en diagonale.

La ligne à écrire : `#include <deplace>`.

Ses arguments : colonne, ligne, tuile, `pasX`, `pasY`. Seule sur sa ligne, elle range l'arrivée dans les deux variables.

Ce qu'elle coûte : environ **300 octets**, plus la police entière.

Essaie : `deplace(x, y, ALPHABET[0], 6, 0);` : tout droit.

On retrouve `deplace` dans 9 leçons

[0.26](#), [0.26.1](#), [0.26.2](#), [0.27](#), [0.27.1](#), [0.27.2](#), [0.32](#), [0.32.1](#), [0.32.2](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1158 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	64 o	← la console, à l'allumage – la mise en route de la console
<code>deplace()</code>	236 o	← <code>main()</code> – une fonction fournie par le langage
<code>main()</code>	36 o	← démarrage – ta fonction
<code>VBlank</code>	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
<code>AttendreImage</code>	28 o	← <code>image()</code> – attend l'image suivante
<code>AttendreVBlank</code>	16 o	← démarrage – attend que l'écran soit libre
<code>AttendreAcces</code>	12 o	← <code>EffacerCases</code> – attend de pouvoir écrire une case
<code>AdresseCase</code>	10 o	← <code>poser()</code> , <code>effacer()</code> – trouve l'adresse d'une case de l'écran
<code>EffacerCases</code>	12 o	← <code>effacer()</code> – efface des cases
<code>CopierTuiles</code>	18 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
<code>EffacerCarte</code>	14 o	← démarrage – vide les deux cartes de l'écran
<code>Tuiles</code>	704 o	← <code>CopierTuiles</code> – la police entière

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

<code>poser()</code>	× 2	26 o sur place, dans <code>deplace()</code> + <code>AdresseCase</code> (10 o)
<code>effacer()</code>	× 1	15 o sur place, dans <code>deplace()</code> + <code>AdresseCase</code> (10 o), <code>EffacerCases</code> (12 o)
<code>image()</code>	× 2	6 o sur place, dans <code>deplace()</code> , <code>main()</code> + <code>AttendreImage</code> (28 o)

```
Tes fonctions appelées :  
deplace() x 1    23 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <deplace> : avancer sur les deux axes ----  
// De (2, 2), 6 pas à droite et 6 en bas : en diagonale, jusqu'en (8, 8).  
  
#include <deplace>    // deplace() : les deux axes à la fois  
#include <ALPHABET>
```

2. UNE VARIABLE — X

```
uint8_t x = 2;
```

3. UNE VARIABLE — Y

```
uint8_t y = 2;
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

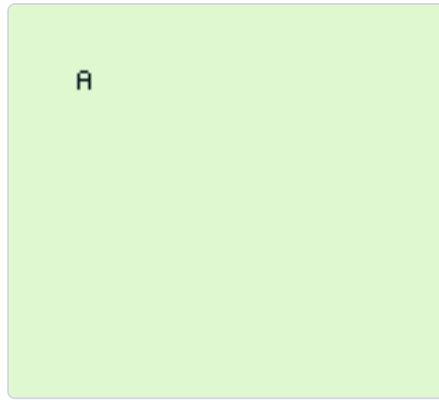
```
int main() {  
    deplace(x, y, ALPHABET[0], 6, 6);    // x et y reçoivent 8 et 8  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

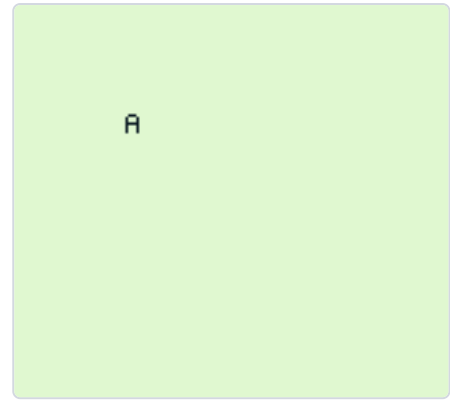
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A qui descend en diagonale vers le milieu de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A passe par (5, 5) : en diagonale
- ✓ il finit en (8, 8) (8,8)
- ✓ x et y ont reçu l'arrivée

Ensuite : **0.26. En diagonale : deplace** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++